ECSE415 - Introduction to Computer Vision

Tutorial 6 - Panorama and image stitching with OpenCV

Announcements

Assignment 2 is dropping very soon (between today and tomorrow).

This tutorial will cover the first part of Assignment 2: Image Stitching.

Quiz 3 will cover lectures 8, 9, 10, and **12** not 11 (skipped for now).

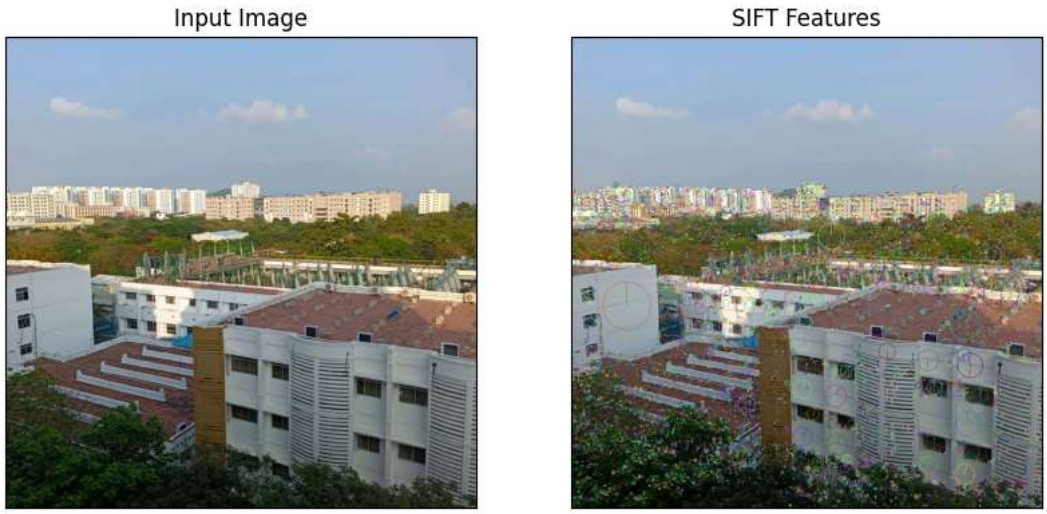
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

from google.colab import drive
drive.mount('/content/drive')

path = '/content/drive/My Drive/Colab Notebooks/assignment2/images/Q1/'
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

SIFT Features and Descriptors

```
#########################
# SIFT FEATURE DETECTOR #
#########################
import cv2
import numpy as np
import matplotlib.pyplot as plt

# load image
img = cv2.imread(path+"view1.jpeg")

# copy image
img_disp = img.copy()

# convert to RGB
img_disp = cv2.cvtColor(img_disp, cv2.COLOR_BGR2RGB)

# create a sift detector
sift = cv2.SIFT_create()

# detect SIFT features, with no masks
# Ref: https://docs.opencv.org/master/d0/d13/classcv_1_1Feature2D.html#aa4e9a7082ec61ebc108806704fbd7887
# SIFT::DETECT(IMAGE, MAKS)
# Image --> Image for which we want to detect SIFT points
# mask -->  Mask specifying where to look for keypoints (optional). It must be a 8-bit integer matrix with non-zero values in the
keypoints = sift.detect(img, None)

# draw the keypoints
# Ref: https://docs.opencv.org/3.4/d4/d5d/group__features2d__draw.html#gab958f8900dd10f14316521c149a60433
# output_image = cv2.drawKeypoints(image, keypoints, outImage, color, flags)
#         cv2.DRAW_MATCHES_FLAGS_DEFAULT: Draws rich keypoints (with size and orientation).
#         cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS: Draws keypoints with their orientation and size shown as circles.
#         cv2.DRAW_MATCHES_FLAGS_DRAW_OVER_OUTIMG: Draws keypoints on the existing image without modifying it.
#         cv2.DRAW_MATCHES_FLAGS_NOT_DRAW_SINGLE_POINTS: Does not draw single keypoints.
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
cv2.drawKeypoints(img, keypoints, img_disp, flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)

# display
plt.figure(figsize=(10,10))
plt.subplot(121), plt.imshow(img)
plt.title("Input Image"), plt.xticks([]), plt.yticks([])
plt.subplot(122), plt.imshow(img_disp)
plt.title("SIFT Features"), plt.xticks([]), plt.yticks([])
plt.show()
```

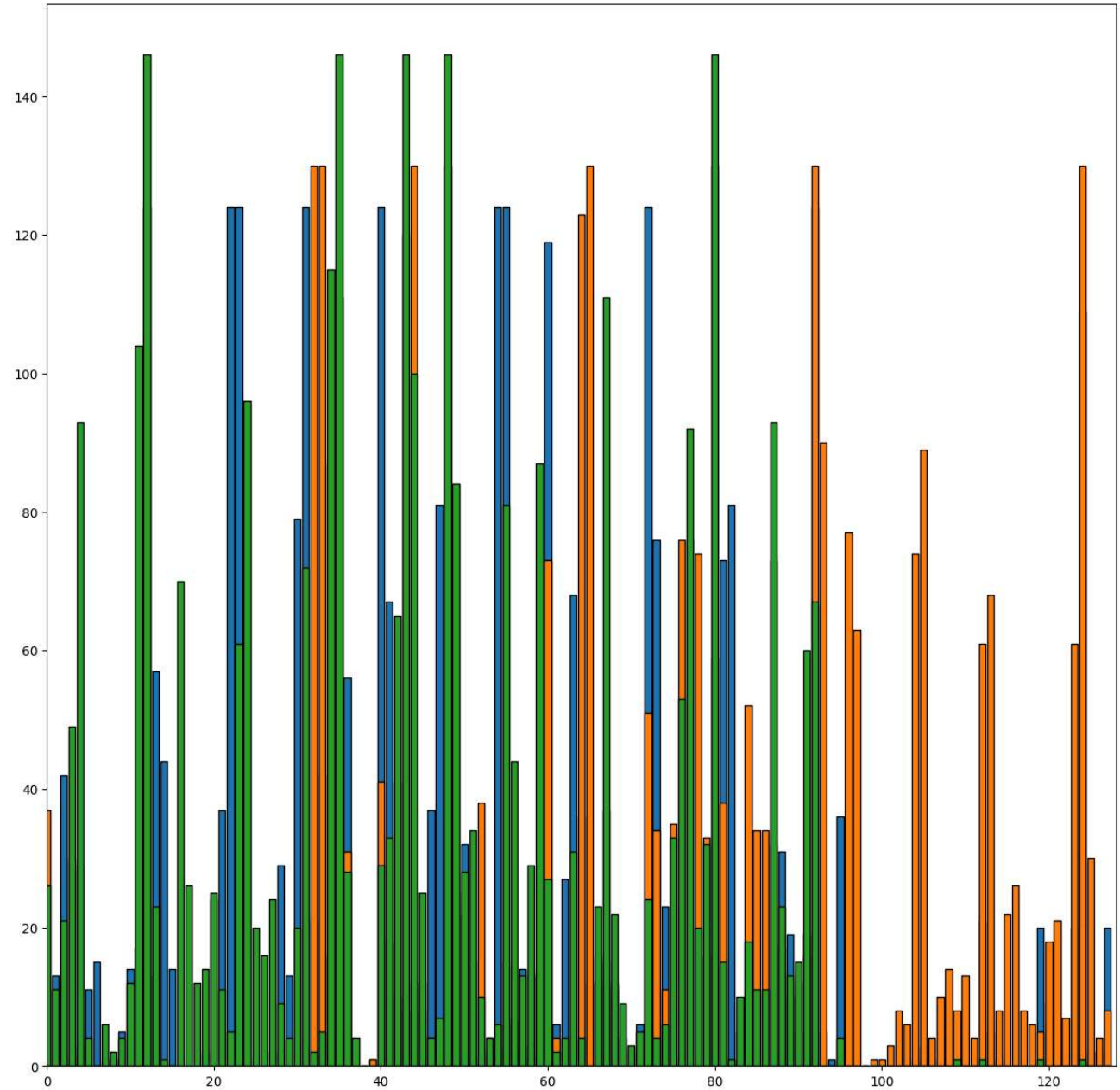
```
plt.show()

# num of SIFT keypoints
print('Num keypoints: '+ str(len(keypoints)))
```




```
Num keypoints: 12581
```

```
####################
# SIFT DESCRIPTORS #
####################

# load image
img = cv2.imread(path+"view1.jpeg")

# create a SIFT object
sift = cv2.SIFT_create()

# detect SIFT features, with no masks
keypoints = sift.detect(img, None)

# compute SIFT descriptors
keypoints, descriptors = sift.compute(img, keypoints)

# Alternatively, we can use keypoints, descriptors = sift.detectAndCompute(gray, None) to get both

# num of SIFT keypoints/descriptors
# Shape: (number of keypoints, 128)
print('descriptors size: '+ str(descriptors.shape))

# plot a couple of 1x128 SIFT descriptors
num = 3
plt.figure(figsize=(15,15))
for i in range(num):
    # plot the 128x1 np array using matplotlib plot()
    # plt.plot(descriptors[i], marker='o', linestyle='-')
    plt.bar(np.arange(0, 128), descriptors[i], edgecolor='k')
    plt.xlim([0,128]) # set x-axis limit
plt.show()
```

```
descriptors size: (12581, 128)
```




```
####################
# FEATURE MATCHING #
####################

# load the images
img1 = cv2.imread(path+"view1.jpeg")
img1 = cv2.cvtColor(img1, cv2.COLOR_BGR2RGB)

img2 = cv2.imread(path+"view2.jpeg")
img2 = cv2.cvtColor(img2, cv2.COLOR_BGR2RGB)
```

```
# Create a SIFT object
sift = cv2.SIFT_create()

# Compute the keypoints/descriptors
keypoints1, descriptors1 = sift.detectAndCompute(img1, None)
keypoints2, descriptors2 = sift.detectAndCompute(img2, None)

# create BFMatcher object
# bf = cv2.BFMatcher(normType = NORM_L2, crossCheck=False)
# normTyp --> One of NORM_L1, NORM_L2, NORM_HAMMING, NORM_HAMMING2.
#             L1 and L2 norms are preferable choices for SIFT and SURF descriptors,
#             NORM_HAMMING should be used with ORB, BRISK and BRIEF, NORM_HAMMING2 should be used with
#             ORB when WTA_K==3 or 4 (see ORB::ORB constructor description).
# crossCheck --> If it is false, this is will be default BFMatcher behaviour when it finds the k nearest neighbors for each que
#                If crossCheck==true, then the knnMatch() method with k=1 will only return pairs (i,j) such that for i-th query
#                j-th descriptor in the matcher's collection is the nearest and vice versa, i.e. the BFMatcher will only return
#                Such technique usually produces best results with minimal number of outliers when there are enough matches. Th
# Ref: https://docs.opencv.org/3.4/d3/da1/classcv_1_1BFMatcher.html#ac6418c6f87e0e12a88979ea57980c020
bf = cv2.BFMatcher(cv2.NORM_L2, crossCheck=True)

# Match descriptors.
matches = bf.match(descriptors1, descriptors2) # query image, train image

# Sort them in the order of their distance.
matches = sorted(matches, key = lambda x:x.distance)

# Draw first 10 matches.
img3 = cv2.drawMatches(img1, keypoints1, img2,keypoints2, matches[:10], None, flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)

plt.figure(figsize=(20,20))
plt.imshow(img3)
plt.title("Matched Keypoints"), plt.xticks([]), plt.yticks([])
plt.show()
```

Matched Keypoints

Image Stitching

```
####################
# IMAGE STITCHING  #
####################

# Read images to stitch
imgl = cv2.imread(path+'view1.jpeg')
imgr = cv2.imread(path+'view2.jpeg')

imgl = cv2.cvtColor(imgl, cv2.COLOR_BGR2RGB)
imgr = cv2.cvtColor(imgr, cv2.COLOR_BGR2RGB)

# Compute keypoints and descriptors
sift_obj = cv2.SIFT_create()
keypoints_l, descriptors_l = sift_obj.detectAndCompute(imgl, None)
keypoints_r, descriptors_r = sift_obj.detectAndCompute(imgr, None)

# create BFMatcher object
bf = cv2.BFMatcher(cv2.NORM_L2, crossCheck=True)

# Match descriptors.
matches = bf.match(descriptors_r, descriptors_l) # query image, train image

# Sort them in the order of their distance.
matches = sorted(matches, key = lambda x:x.distance)

# Draw keypoints
imgmatch = cv2.drawMatches(imgr, keypoints_r, imgl,keypoints_l, matches[:10], None, flags=2)

# Arrange matching keypoints in two separate lists
GoodMatches = []
for i, m in enumerate(matches):
    if m.distance < 1000:
        GoodMatches.append((m.trainIdx, m.queryIdx))

mpr = np.float32([keypoints_r[i].pt for (__, i) in GoodMatches])
mpl = np.float32([keypoints_l[i].pt for (i, __) in GoodMatches])

# find homography and wrap image accordingly
# cv.FindHomography(srcPoints, dstPoints, H, method=0, ransacReprojThreshold=3.0, status=None)
# Parameters:
# srcPoints – Coordinates of the points in the original plane, a matrix of the type CV_32FC2 or vector<Point2f> .
# dstPoints – Coordinates of the points in the target plane, a matrix of the type CV_32FC2 or a vector<Point2f> .
# method – Method used to computed a homography matrix. The following methods are possible:
#          0 - a regular method using all the points
#          CV_RANSAC - RANSAC-based robust method
#          CV_LMEDS - Least-Median robust method
# ransacReprojThreshold – Maximum allowed reprojection error to treat a point pair as an inlier (used in the RANSAC method only)
#                         | dstPoints_i -  convertPointsHomogeneous( H * srcPoints_i) |  >  ransacReprojThreshold
#                         then the point i is considered an outlier. If srcPoints and dstPoints are measured in pixels, it usual
# mask – Optional output mask set by a robust method ( CV_RANSAC or CV_LMEDS ). Note that the input mask values are ignored.
# Ref: https://docs.opencv.org/2.4/modules/calib3d/doc/camera_calibration_and_3d_reconstruction.html#findhomography
H, __ = cv2.findHomography(mpr, mpl, cv2.RANSAC, 4)

# Ref: https://docs.opencv.org/master/da/d54/group__imgproc__transform.html#gaf73673a7e8e18ec6963e3774e6a94b87
# dst = cv2.warpPerspective(src, M, dsize, flags, boaderMode, boarderValue)
# src --> input image.
# dst --> output image that has the size dsize and the same type as src .
# M --> 3×3 transformation matrix.
# dsize --> size of the output image.
# flags --> combination of interpolation methods (INTER_LINEAR or INTER_NEAREST) and the optional flag WARP_INVERSE_MAP, that se
# borderMode --> pixel extrapolation method (BORDER_CONSTANT or BORDER_REPLICATE).
# borderValue --> value used in case of a constant border; by default, it equals 0.
wimg = cv2.warpPerspective(imgr, H, (imgr.shape[1]+imgl.shape[1], imgr.shape[0]))
wimg[:, :imgl.shape[1], :] = imgl

plt.figure(figsize=(20,20))
plt.subplot(211)
plt.imshow(imgmatch)
plt.title("Matched Keypoints"), plt.xticks([]), plt.yticks([])
plt.subplot(212)
plt.imshow(wimg)
plt.title("Panoramic image"), plt.xticks([]), plt.yticks([])
plt.show()
```